

LiDAR Lenses Wave — Camera Studio

A single-file web front-end that drives the **real** LiDAR Lenses Wave engine from a browser. Pick a built-in scene or upload your own 3D model, choose a sensor preset, and it runs the full multimodal pipeline — depth, optical/acoustic coherence, ultrasonic, polarization, material classification, boundary-aware edges — then shows you the engine's actual diagnostic contact sheet and reports.

Unlike a lightweight JS re-implementation, this front-end makes the same `run_sensor_preset()` call the engine's own test harnesses use. What you see in the browser is what the engine actually produced.

It exists for two jobs:

- **Inspect and correct 3D scenes.** Render a generated mesh back through the engine to check whether the geometry came out the way you intended — scale, framing, holes, silhouette, solidity — and iterate.
 - **Showcase the engine.** Give people a hands-on way to see what the engine does without touching the code.
-

What you get back

A scan returns a contact sheet plus several stand-alone panels and reports:

- **Shaded** — a plain rendered view of the scene.
- **Depth** — per-pixel range (catches floaters, wrong scale, gaps).
- **light_anti / sound_anti** — optical and acoustic anti-coherence channels.
- **edge_score_raw** → **edge_score_geom** → **edge_confidence** — the edge detector going from raw evidence to geometry-filtered boundaries to a confidence map.
- **geom_edge overlay** — boundaries drawn over the shaded view.
- **Classification** — per-pixel labels (`solid_surface`, `foliage`, `geom_edge`, `partial_occluder`, `sky`, ...) with counts.
- A **run summary** (resolution, lens, camera, coverage, depth percentiles, runtime), a **material channel report**, and any **warnings**.

Note: several of the "wave" channels are derived from the same depth buffer and material priors, so they correlate heavily with one another. They are useful as visual cues and for scene inspection; they are not independent physical sensors.

Requirements

- **Python 3.8+**
- **NumPy and Pillow** (`pip install numpy Pillow`)
- **A LiDAR Lenses Wave engine** (`.py`) file (e.g. `lidar_lenses_wave_v070.py`). The engine itself is the only thing that pulls in numpy/Pillow; the front-end is pure standard library otherwise.

No web framework, no third-party JS — everything (server + page) is one file.

Quick start

Put the studio file next to your engine and run it:

```
bash

python lidar_studio_camera.py
```

Then open `http://localhost:8080` in a browser.

By default the studio looks for an engine automatically (the v0.8 alpha path, then `lidar_lenses_wave_v070.py` in the same folder). To point at a specific one:

```
bash

python lidar_studio_camera.py --engine lidar_lenses_wave_v070.py --port 8080
```

If no engine is found and engine-loading is enabled, you can also load one from the page — upload an engine (`.py`) or give a path on the server.

Command-line options

Flag	Default	Purpose
<code>--engine PATH</code>	auto-detect	Path to the engine (<code>.py</code>) to load at startup.
<code>--host HOST</code>	<code>127.0.0.1</code>	Address to bind. Non-loopback requires <code>--allow-remote</code> .
<code>--port PORT</code>	<code>8080</code>	Port to serve on.

Flag	Default	Purpose
<code>--allow-remote</code>	off	Permit binding to a non-loopback (network-exposed) host.
<code>--allow-engine-load</code>	off*	Enable the engine load/upload endpoints on a remote bind.
<code>--max-upload-mb N</code>	<code>200</code>	Maximum request body size, in MB.

* Engine-loading is always enabled on a loopback bind (the local user already has a shell). On a remote bind it is **off** unless you pass `--allow-engine-load`.

Using it

Scenes. Three options in the panel:

- *Cabin demo* — the engine's built-in procedural log-cabin scene.
- *Material board* — the built-in material target board.
- *Upload model* — your own `.stl` or `.obj`.

Uploads. Binary or ASCII `.stl` (via the engine's native loader) or a triangulated Wavefront `.obj` (via a small built-in parser). On upload you can:

- assign a **material assumption** (drives acoustic/ultrasonic/polarization priors; `auto` = neutral mesh defaults),
- **normalize** the model (recenter, drop to the ground plane, scale to a ~6 m footprint so it fits the presets' range assumptions),
- **simplify** heavy meshes — anything above the *max triangles* budget is decimated by vertex clustering before scanning (heavy meshes are slow to raycast).

Presets. The dropdown is populated from whatever engine is loaded. The v0.7.0 engine ships: `compact_diagnostic`, `full_diagnostic`, `edge_debug`, `indoor_structure`, `material_scan`, `outdoor_occlusion`, `overhead_layout`.

Fast preview vs. high quality. Fast preview (default, recommended for heavy meshes) skips the engine's slow auto-framer and burst stacking, substituting a cheap framing search — every channel is still produced, just single-burst and a bit noisier. Unchecking it runs the full auto-framed, multi-burst render, which is much slower (tens of seconds on a heavy mesh vs. a few seconds in fast mode).

Manual camera / lens. Enable it to send an exact camera to the engine (and disable preset auto-framing): choose a lens (pinhole / telephoto / fisheye / orthographic / equirectangular), FOV / fisheye FOV / ortho size, and explicit position, target, and up vectors. Use this when you want the *same* view across several scene revisions so you can compare them.

How it works

The studio loads the engine module at runtime and applies a small, additive compatibility shim (`patch_engine`) so an uploaded mesh flows through the **same** diagnostics pipeline as the built-in primitive scenes. The engine already raycasts meshes (BVH + Möller-Trumbore) and `Scene` already holds a `meshes` list; the shim just makes `Scene` iterable/sized and teaches camera auto-framing about mesh AABBs. The engine source files are never modified.

For fast previews, the engine's auto-framer (the dominant cost) is replaced by a handful of low-resolution candidate bursts on a ring around the scene; the framing that fills the frame without clipping is handed to the engine with auto-framing off.

The server is multi-threaded for responsiveness, but a lock serializes the actual engine work — scans run one at a time, while the page, `/meta`, and `/health` stay responsive.

HTTP endpoints

- `GET /` — the page
 - `GET /meta` — engine status + preset/material option lists
 - `GET /health` — liveness + which engine is loaded
 - `POST /scan` — run a scan (JSON request)
 - `POST /load_engine` — load an engine by server path (*see security note*)
 - `POST /upload_engine` — upload and load an engine `.py` (*see security note*)
-

Security

The engine load/upload endpoints **execute arbitrary Python by design** (loading an engine module runs its top-level code). That is fine for a single local user — you already have a shell — but it is a remote-code-execution hole the moment the server is reachable on a network. The studio therefore defaults to safe:

- It **refuses to bind to a non-loopback host** unless you pass `--allow-remote`.
- On a remote bind, the engine load/upload endpoints stay **disabled** unless you also pass `--allow-engine-load`; the UI hides the loader and explains why.

- Request bodies are **capped** (`--max-upload-mb`, default 200) so a client can't exhaust memory just by claiming a huge `Content-Length`.

Even with both flags set, only expose this to networks and users you trust.

Notes & limitations

- Browser scans are clamped for responsiveness: width 64-800, height 64-600, 1-16 rays/pixel.
- The OBJ parser handles `(v)`/`(f)` lines (fan-triangulating polygons, with bounds checks); texture/normal indices are ignored.
- On a remote bind without `--allow-engine-load`, you must start with `--engine PATH` since the UI can't load one.
- This is a front-end. It needs a compatible engine exposing `SENSOR_PRESETS`, `run_sensor_preset`, `Scene`, and `Mesh`; it validates this when loading and reports clearly if a file doesn't look like a LiDAR Lenses Wave engine.